

Module TS Wording Does Not Support Intended Use Case

- Document number: P0832R0
- Date: 2016-10-14
- Reply-to: Steve Downey <sdowney2@bloomberg.net>, David Sankel <dsankel@bloomberg.net>
- Audience: Evolution/Core

Abstract

Although we believe the model for modules as proposed is fundamentally sound, the wording still lacks a means to convert a very large codebase to use modules in a manner that is additive. That is, the conversion process must not require substantive rewrites of existing code, as that would likely introduce errors. Exposing existing code as a module should involve providing a new module interface unit that can reuse existing source files without modification such that current consumers are not broken.

Core Use Case

Bloomberg has a very large existing base of C++ code. We do not believe we are unique, and we do believe our concerns are shared by any entity with a large existing investment in C++.

Background

1. Over 250,000 translation units
2. The same code is built on 4 major operating systems using 4 different compilers
3. Atomic changes across the codebase are not possible

Business Requirements

1. Core services must always build
2. Breaking non-core services is discouraged
3. Both bug fixes and features are delivered at tip of tree, not to old versions

This has several implications for creating a module for an existing facility. The pre-existing header must still work. It must be possible to use both the existing header and the module in one program. It must also be possible to use the header and the module in a single translation unit, mentioning them in either order.

Creating a new facility exposed by modules does not have the same issues since there will be no pre-existing consumers. However, completely new development is much rarer than extending existing facilities, and the largest benefits will be from modularizing the most widely re-used components.

Expected Solution

Export with Using Declarations and Using Directives

From discussion at previous meetings, we believe this is the intended migration path for transitioning to modules while retaining compatibility with existing headers.

```

// facility.h
#include <other_library.h>

namespace facility {
class Foo {};
int func(Foo const& foo);
int func(Foo&& foo);
enum BAR {ONE, TWO, THREE};
// ...
}

// facility module interface unit
#include <facility.h> // outside the purview of module facility

export module facility;

namespace facility {
export using Foo = facility::Foo;
export using func = facility::func;
export using BAR = facility::BAR;
}

// main.cpp
import facility;

int main()
{
    facility::Foo f;
    int i = facility::func(f);
    facility::BAR b = facility::ONE;
    return i;
}

```

The facility has an implementation dependency on some other library (`other_library`), whose header is `other_library.h`. We include `facility.h` outside the purview of the module, and we explicitly export the names we want exposed from the facility namespace, excluding any names from `other_library`. It ensures that the entities are not owned by the module, so there are no ODR issues if `facility` is both included and imported into the same translation unit, or if used by both methods in the same program.

The new module interface can be imported by current users of the facility header. If these users were dependent, either by accident or design, on something that is not exported by the module, they can discover this at build time. They can then negotiate with the owners of `facility` to get what they need exposed, or they can import or include from the appropriate source. Users, at any rate, are no longer exposed to any names in transitive includes; They must be explicit about their dependencies.

This, unfortunately, does not appear to work according to the language in the draft TS. Only entities with linkage may be exported, according to `[dcl.module.interface]`. Using declarations and directives are not entities with linkage.

This leads us to a bigger issue. None of using declarations, using directives, typedefs, namespace aliases, deduction guides, nor macros are entities. Additionally, there are entities that do not have linkage: alias templates, and enumerators. None of these can be exported.

The omission of typedefs, and related features, from the set of entities (as previously defined in the standard) is intentional. These are mechanisms for giving names to existing entities without creating new entities. In the Standard `[basic]p3` lists entities, while in `[basic]p5`, declarations may introduce additional names for an entity.

Alternative Approaches

In this section we outline a few alternative approaches for exposing an existing component as a module, some of which do not comply with our core use-case. Before we get into these approaches, we state our assumptions about how modules would be implemented in practice.

Assumptions about how Modules work

There are some underlying assumptions about the concrete implementation of modules that are outside what is specified by the standard or proposal. We are calling these out specifically because if these are incorrect, it is likely that the alternative approaches are not meaningful.

1. Compiling a module interface unit produces a file to be consumed by the compiler when it encounters a `import` directive
2. The result of compiling a module interface is not portable
 - Between compilers
 - Between the same compiler using different std libraries
 - Between the same compiler using different options, such as threading, exceptions, etc.
3. The build system will have to compile the module interface unit first, before it can be used as an import by the compiler, much like a library to the linker
4. The linker does not use the result of compiling the module interface unit
5. Libraries exporting symbols in named modules and the global module are identical
6. There is no change in name-mangling due to modules
7. Module linkage is internal to the compiler, not part of the binary interface
8. The compiler is not a build system, and does not compile the module interface unit upon finding an `import` directive

Complete Rewrite

Although a complete rewrite of an existing codebase in terms of modules is the cleanest solution, it is generally ruled out by our business requirements. Existing consumers must not be broken, and need to be provided with any bug fixes. Providing them with an older version is not an acceptable solution. The approach could only be used by teams not providing code to other groups.

Exporting the Included Header

```
export module facility;
export {
#include <facility.h>
}
```

This provides poor control over the visibility of parts of the facility, and inappropriately exports any headers that `facility.h` `#includes`, such as standard library headers. This would violate the rule that there is exactly one module that owns an entity. Although this could possibly be prevented by duplicating the includes in the module interface unit outside the module purview it would come with a significant ongoing maintenance cost.

```
#include "dependency.h"

#include <vector>
#include <string>

export module facility;
export {
```

```
#include <facility.h>
}
```

Import the module in the header

```
// facility.h

#ifdef __cpp_modules
import facility;
#else
#include "facility_impl.h"
#endif
/* Provide MACROS etc */
```

The header `facility_impl.h` can be a subset of the original header and use a conditionally defined macro to control export from the module interface unit.

```
//facility_impl.h
namespace facility {
FACILITY_EXPORT class Foo {
    // ...
};

FACILITY_EXPORT int func(Foo const&);
FACILITY_EXPORT int func(Foo&&);

FACILITY_EXPORT enum BAR {ONE, TWO, THREE};
}

//facility module interface
export module facility;

#define FACILITY_EXPORT export
#include "facility_impl.h"
```

This approach has the disadvantage of a fairly invasive rewrite of the facility header to split out the parts that can be modularized and the parts that can not, such as macros, as well as separating out parts that should not be part of the module's interface, such as a `facility::detail` namespace. It will, however, allow mixed use of the module and header assuming there is no differences in the symbols produced by the module implementation unit. One must also consider the possibility that someone will use `facility_impl.h` directly.

This approach also pushes changes to users of the facility header, possibly breaking them, which violates our business rules.

Parallel headers separated by filesystem

The modularized headers could be maintained as separate files that live in a different location in the filesystem. Compilers that can consume modules could have their `-I` flags adjusted to make those visible.

This has the disadvantage of requiring either maintenance of two versions of the library or extensive rewrites of the existing headers so that they can be shared.

Conclusion

The choice of syntax is not critical, but the capability of modularizing an existing codebase while maintaining compatibility with existing users of a facility without risky rewrites is. It must be possible to expose existing code through a module by strictly additive methods. The modules TS as drafted does not support this.

References

1. N4660, Working Draft, Standard for Programming Language C++
2. Programming Languages — Extensions to C++ for Modules, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4689.pdf>
3. Business Requirements for Modules, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0678r0.pdf>